

Graph Traversal: BFS and DFS

Programming and Data Structures — Week 11, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain precisely why graph traversal requires an explicit “visited” set, a requirement Week 7’s tree traversal never needed; implement breadth-first search using Week 3’s queue, and trace it completely on a concrete graph; implement depth-first search using Week 3’s stack, in both its explicit- stack and recursive call-stack forms, and trace it completely; and state precisely which class of real-world question each traversal answers most naturally.

Outline

Today covers a maze-exploration analogy motivating two different search strategies; why graphs specifically require an explicit visited set, a genuine departure from Week 7’s tree traversal; breadth-first search, built directly on Week 3’s queue, traced completely; depth-first search, built directly on Week 3’s stack, traced completely; which real-world question each traversal answers most naturally; a precise justification of both traversals’ shared $O(V+E)$ cost; the MTech-track material on DFS’s recursive form and its call-stack depth risk; an in-class quiz; and a summary.

The maze-exploration analogy

Picture exploring a maze two different ways. Breadth-first search explores like ripples spreading outward from a dropped stone: check every room exactly 1 step away first, then every room exactly 2 steps away, and so on, expanding outward one full layer at a time. Depth-first search explores like a person committed to following one corridor as far as it possibly goes, only backtracking to try an alternative when a dead end is reached. Both strategies eventually visit every room reachable from the starting point — they differ entirely in the order rooms are visited, not in which rooms are eventually reached.

Why graphs need a “visited” set

Week 7’s tree traversal algorithms never needed to track which nodes had already been visited, because a tree by definition has no cycles — no node is ever reachable more than once from the root via distinct paths. A general graph can have cycles, exactly Week 11 Lecture 1’s mutual- friend-group example, and without explicitly tracking visited nodes, a traversal could revisit the same nodes indefinitely, looping forever and never terminating. This single addition — a `visited` set, checked before processing any node — is the entire structural difference separating this lecture’s graph traversal from Week 7’s tree traversal.

BFS, built on Week 3’s queue

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()          # Week 3's Queue: FIFO
        order.append(node)
        for neighbor, weight in graph.adj[node]:
            if neighbor not in visited:
                visited.add(neighbor)  # mark visited when ENQUEUED, not when dequeued
                queue.append(neighbor)
    return order
```

BFS is built directly on Week 3’s queue: neighbors are enqueued as they are discovered, and nodes are dequeued and processed in strict FIFO order, guaranteeing the “expand outward one layer at a time” behavior described in the maze analogy. Marking a node as visited at the moment it is enqueued, rather than waiting until it is dequeued and processed, is essential — without this, the same node could be discovered and enqueued multiple times by different neighbors before it is ever processed, wasting work and potentially producing an incorrect traversal order.

Tracing BFS completely on Lecture 1’s example graph

Graph: `adj[0]=[1,2]`, `adj[1]=[0,2,3]`, `adj[2]=[0,1]`, `adj[3]=[1,4]`, `adj[4]=[3]`
`bfs(graph, 0):`

`visited={0}`, `queue=[0]`

Dequeue 0. order=[0]. Neighbors 1,2: both unvisited → visited={0,1,2}, queue=[1,2]
 Dequeue 1. order=[0,1]. Neighbors 0(visited),2(visited),3(new) → visited={0,1,2,3}, queue=[2]
 Dequeue 2. order=[0,1,2]. Neighbors 0,1 both visited → queue=[3]
 Dequeue 3. order=[0,1,2,3]. Neighbor 1(visited), 4(new) → visited={0,1,2,3,4}, queue=[4]
 Dequeue 4. order=[0,1,2,3,4]. Neighbor 3(visited) → queue=[]

Final BFS order: [0, 1, 2, 3, 4]

Tracing BFS completely on Lecture 1's 5-node example graph, starting from node 0: the traversal visits 0 first, then its immediate neighbors 1 and 2 (layer 1), then node 3, reached through node 1 (layer 2), and finally node 4, reached through node 3 (layer 3). The resulting order, [0, 1, 2, 3, 4], correctly reflects increasing distance from the starting node — exactly the “ripples spreading outward” behavior described in the maze analogy, made concrete on this specific graph.

DFS, built on Week 3's stack

```
def dfs(graph, start):
    visited = set()
    stack = [start]
    order = []
    while stack:
        node = stack.pop()                # Week 3's Stack: LIFO
        if node in visited:
            continue
        visited.add(node)                 # mark visited when POPPED, not when pushed
        order.append(node)
        for neighbor, weight in graph.adj[node]:
            if neighbor not in visited:
                stack.append(neighbor)
    return order
```

DFS is built directly on Week 3's stack: neighbors are pushed as discovered, and nodes are popped and processed in strict LIFO order, producing the “follow one corridor as far as possible” behavior from the maze analogy. Unlike BFS's implementation, this version marks `visited` at pop time rather than push time, which means a node can legitimately be pushed onto the stack multiple times before its first processing; the `if node in visited: continue` check simply skips any duplicate encountered later, a valid and common way to implement iterative DFS.

Tracing DFS completely on the same graph

```
dfs(graph, 0): adj[0]=[1,2], adj[1]=[0,2,3], adj[2]=[0,1], adj[3]=[1,4], adj[4]=[3]
```

```
stack=[0], visited={}
```

```
Pop 0. Not visited. visited={0}. order=[0]. Push 1, then 2 (unvisited) → stack=[1,2]
```

```
Pop 2 (top of stack). Not visited. visited={0,2}. order=[0,2]. Push 1 (0 already visited) → stack=[1]
```

```
Pop 1 (top). Not visited. visited={0,2,1}. order=[0,2,1]. Push 3 (0,2 already visited) → stack=[3]
```

```
Pop 3 (top). Not visited. visited={0,2,1,3}. order=[0,2,1,3]. Push 4 (1 already visited) → stack=[4]
```

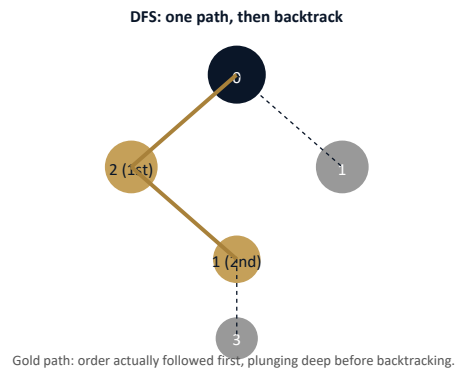
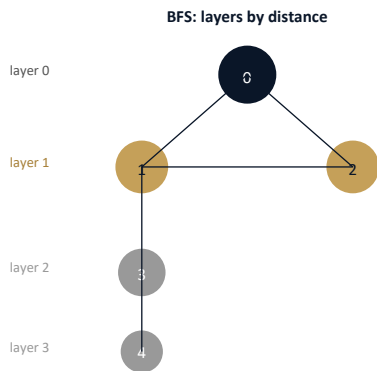
```
Pop 4 (top). Not visited. visited={0,2,1,3,4}. order=[0,2,1,3,4]. 3 already visited, nothing to push
```

```
Pop 1. Already visited → skip. stack empty.
```

Final DFS order: [0, 2, 1, 3, 4]

Tracing DFS on the identical graph, starting from node 0: because the stack is LIFO, the *last*-pushed neighbor is explored first — node 0 pushes 1 then 2, so 2 is popped and processed first, diverging immediately from BFS's order. The traversal then follows node 2 to node 1, node 1 to node 3, and node 3 to node 4, producing the final order [0, 2, 1, 3, 4] — visibly different from BFS's [0, 1, 2, 3, 4] on the identical graph, demonstrating concretely that traversal order genuinely depends on the strategy chosen, not merely on the graph's structure.

The picture: BFS layers vs. DFS depth-first plunge



BFS's visitation order visibly groups by distance from the start node, forming concentric layers; DFS's visitation order instead traces a single long path before backtracking, visually resembling depth rather than breadth — exactly the property each algorithm's name describes.

BFS vs. DFS: which answers which question

BFS naturally finds the shortest path, measured in number of edges, in an unweighted graph, precisely because it explores nodes in strict order of increasing distance from the start — the first time a target node is reached, that path is guaranteed shortest. DFS naturally answers “does any path exist at all” and forms the natural basis for detecting cycles and for topological sorting, both covered in Lecture 3. Neither traversal is unconditionally better — this is Week 8 Lecture 4’s structure-choice lesson recurring, now applied to a choice of traversal *strategy* rather than storage structure, with the correct choice depending entirely on which question is actually being asked.

Both traversals’ cost: $O(V+E)$

Both traversals share an identical cost analysis. Every node is enqueued or pushed, and dequeued or popped, at most once, guarded by the visited check, giving $O(V)$ total node-processing work. Every edge is examined exactly once per direction while iterating through its endpoint’s neighbor list, giving $O(E)$ total edge-examination work. Summing these gives $O(V + E)$ total cost for both BFS and DFS — they differ entirely in traversal *order*, as this lecture’s traces demonstrated concretely, but never in asymptotic cost.

MTech addition — DFS’s recursive form, and its risk

```
def dfs_recursive(graph, node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for neighbor, weight in graph.adj[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited, order)
    return order
```

DFS has a natural recursive formulation, where the function call stack itself serves as the stack, with no explicit stack variable needed at all — directly connecting back to Week 3’s observation that every recursive function implicitly uses a call stack. The risk: for a graph containing a very long path, the recursion depth equals that path’s length, and Python’s default recursion limit, around 1000, can be exceeded, raising a `RecursionError` for sufficiently deep or large graphs. The iterative version presented in the main lecture slides avoids this risk entirely, at the cost of slightly more explicit bookkeeping — a genuine, concrete trade-off worth knowing, not merely a stylistic preference between two equivalent implementations.

A second worked BFS trace, from a different start node

```
bfs(graph, 4):  adj[0]=[1,2], adj[1]=[0,2,3], adj[2]=[0,1], adj[3]=[1,4], adj[4]=[3]

visited={4}, queue=[4]
Dequeue 4. order=[4]. Neighbor 3 new → visited={4,3}, queue=[3]
Dequeue 3. order=[4,3]. Neighbor 1 new (4 visited) → visited={4,3,1}, queue=[1]
Dequeue 1. order=[4,3,1]. Neighbors 0,2 new (3 visited) → visited={4,3,1,0,2}, queue=[0,2]
Dequeue 0. order=[4,3,1,0]. Neighbors 1,2 both visited → queue=[2]
Dequeue 2. order=[4,3,1,0,2]. Neighbors 0,1 both visited → queue=[]

Final: [4, 3, 1, 0, 2]
```

Tracing BFS from a different starting node, 4, on the identical graph confirms the algorithm correctly adapts to any starting point: the layers now expand outward from 4 instead of 0, giving the order [4, 3, 1, 0, 2] — node 4's single neighbor, 3, is visited first, then 3's neighbor 1, then 1's two remaining unvisited neighbors, 0 and 2. This is worth verifying independently by running the code, not merely trusting the first traced example generalizes automatically.

Exercise

As an exercise, implement both `bfs` and `dfs` exactly as presented in this lecture, and verify them against the traced 5-node example, confirming the exact orders [0, 1, 2, 3, 4] and [0, 2, 1, 3, 4] respectively. Separately, implement `dfs_recursive` and confirm it produces an identical order to the iterative `dfs` version on the same graph, then construct a graph consisting of a single chain of 100 or more nodes and observe the recursive version's behavior as the chain length approaches Python's default recursion limit.

Real-world use of BFS and DFS

BFS underlies “degrees of separation” features on social networking platforms and finding the shortest number of link-hops between two web pages. DFS underlies dependency resolution — installing a software package that requires another, which itself requires a third, is naturally ordered by DFS — as well as maze and puzzle solvers, and detecting deadlock cycles in resource allocation graphs within operating systems. Both traversals appear inside web crawlers, garbage collectors tracing which objects remain reachable, and compilers analyzing a program's structure — genuinely foundational algorithms, not merely textbook exercises.

Quiz — predict the BFS order

Given a graph with $\text{adj}[0]=[1,3]$, $\text{adj}[1]=[0,2]$, $\text{adj}[2]=[1]$, and $\text{adj}[3]=[0]$, determine the BFS visitation order starting from node 0, before checking the answer.

A second worked DFS trace, from a different start node

```
dfs(graph, 4):  adj[0]=[1,2], adj[1]=[0,2,3], adj[2]=[0,1], adj[3]=[1,4], adj[4]=[3]
```

```
stack=[4]
```

```
Pop 4. visited={4}. order=[4]. Push 3 → stack=[3]
```

```
Pop 3. visited={4,3}. order=[4,3]. Push 1 (4 visited) → stack=[1]
```

```
Pop 1. visited={4,3,1}. order=[4,3,1]. Push 0, then 2 (3 visited) → stack=[0,2]
```

```
Pop 2 (top). visited={4,3,1,2}. order=[4,3,1,2]. Push 0 (1 visited) → stack=[0,0]
```

```
Pop 0. visited={4,3,1,2,0}. order=[4,3,1,2,0]. Both neighbors visited → stack=[0]
```

```
Pop 0. Already visited → skip. stack empty.
```

```
Final: [4, 3, 1, 2, 0]
```

Tracing DFS from node 4 confirms the LIFO “last pushed, first explored” behavior once again: after reaching node 1, its two unvisited neighbors, 0 and 2, are pushed in that order, but 2 — pushed last — is explored first, before backtracking to explore 0. The final order, [4, 3, 1, 2, 0], again differs from BFS’s order on the same starting node, [4, 3, 1, 0, 2], differing only in the relative order of the final two nodes — exactly where the two algorithms’ strategies diverge.

Quiz — answer

Starting from node 0, both neighbors 1 and 3 are discovered and enqueued in that order. Dequeueing 1 next discovers neighbor 2, newly enqueued. Dequeueing 3 discovers no new neighbors, since 0 is already visited. Dequeueing 2 discovers no new neighbors either, since 1 is already visited. The final BFS order is [0, 1, 3, 2] — node 2, despite being adjacent to node 1 which was processed early, is processed last, since it is genuinely two steps away from the start, correctly reflecting BFS’s layer-by-layer exploration order.

A closing note on directed graphs and traversal

Both BFS and DFS, exactly as written in this lecture, work unchanged on directed graphs — the traversal simply follows whatever entries `graph.adj[node]` actually contains, regardless of which direction they represent. On a directed graph, however, traversal starting from node u may not reach every node reachable from some other node v , even if v is itself reachable from u — reachability is no longer

guaranteed symmetric, unlike the undirected graphs traced throughout this lecture. This asymmetry is exactly why Week 12's shortest-path algorithms must track edge direction carefully throughout, a detail worth holding onto ahead of that lecture.

Summary

Graph traversal requires an explicit visited set, a genuine departure from Week 7's tree traversal, since a graph's cycles could otherwise cause traversal to loop forever. BFS, built on Week 3's queue, explores nodes in strict order of increasing distance from the start; DFS, built on Week 3's stack, plunges as deep as possible along one path before backtracking on dead ends. Both cost $O(V+E)$, an identical asymptotic result, differing entirely in traversal order and in which real-world questions each answers most naturally — BFS for shortest unweighted paths, DFS for cycle detection and topological sorting. Next lecture builds three concrete applications directly on top of this lecture's two traversal algorithms: finding connected components, detecting cycles, and topological sorting.